

Credit Name: CSE 3130 Object-Oriented Programming 2
Assignment Name: carInfoShop.java

The following questions can help you in thinking critically about your problem-solving processes:

Understanding the Problem

How did you approach understanding the challenge?

I approached understanding the challenge by first breaking down the requirements into smaller parts, focusing on what the program needed to do overall and then identifying the relationships between the classes. I recognized that all vehicles share common attributes like year, model, and make, so I used an abstract superclass to represent those shared features. Then, I analyzed the differences between cars, trucks, and minivans to create subclasses with their own specific properties and behaviors. By thinking about inheritance and how each class would extend the base functionality, I was able to organize the program logically before implementing the menu-driven interaction in the main class.

```
public abstract class vehicle {
```

Were there any parts of the problem you found confusing at first? If so, how did you resolve that confusion?

At first, one confusing part was understanding how the abstract class and the subclasses should interact, especially how the `getDetails()` method works across all vehicle types. It was a bit unclear how each subclass would provide its own version of that method while still using the common information from the vehicle class. I resolved this by reviewing how inheritance and abstract methods work, and by practicing how `super.toString()` can be used to reuse code from the parent class. Testing small parts of the program and seeing how each subclass overrides the method also helped clarify how everything connects together.

```
// abstract method that must be implement  
public abstract String getDetails();
```

```
// combine parent class's  
return super.toString() +
```

Planning the Solution

Did you create a plan or break the problem into smaller steps before coding?

Yes, I broke the problem into smaller steps before starting to code. First, I identified the common features that all vehicles share and created the vehicle superclass. Next, I planned out each subclass (car, truck, and minivan) by deciding what unique attributes they would have and how they would implement the `getDetails()` method. After that, I focused on building and testing each class individually to make sure they worked correctly. Finally, I created the main program with the menu system to bring everything together and handle user input.

A screenshot of an IDE's tab bar showing five open files: vehicle.java, truck.java, car.java, minivan.java, and carInfoShop.java. The first four tabs have a small icon on the left and a close button (X) on the right. The fifth tab, carInfoShop.java, also has a close button.

These are the classes I did it order and when I was making the codes this allowed me to extend to other class so the main class is able to work flawlessly

How did you decide on the tools, data structures, or algorithms to use?

I decided on the tools and structure by focusing on the type of problem and the concepts we were learning in class. Since the program involved different types of vehicles that share common features, I chose inheritance with an abstract superclass to keep the design organized and avoid repeating code. I used simple data types like `int`, `String`, and `boolean` because they were enough to store the required information, and no complex data structures were needed. For the program flow, I used a loop and conditional statements to handle the menu system, since that is a straightforward and efficient way to process user input and display the correct details.

```
private int seat  
  
// indicates whe  
private boolean
```

Implementation

Did you write the code in small pieces or attempt the entire solution at once?

I wrote the code in small pieces rather than attempting the entire solution at once. I started by creating the vehicle superclass and making sure it worked properly, then moved on to each subclass (car, truck, and minivan) one at a time. After confirming that each class functioned correctly on its own, I combined everything in the main program and added the menu system. This step-by-step approach made it easier to find and fix errors and ensured that each part was working before moving on.

How did you test your solution along the way to make sure it was working?

I tested my solution step by step as I built each part of the program. After creating each class, I would run small tests by creating objects and calling their `getDetails()` method to make sure the output was correct. I also checked that the `toString()` method from the superclass was working properly with the subclasses. Once everything was combined, I tested the full program by running the menu multiple times and trying different inputs (C, T, M, and Q) to make sure it displayed the correct details and looped as expected. This helped me catch and fix any issues early before they became bigger problems.

Overcoming Challenges

What part of the problem was the most difficult for you?

The most difficult part of the problem was properly using inheritance and making sure each subclass correctly extended the vehicle class. It was challenging to understand how to structure the classes so that common information was stored in the superclass while still allowing each subclass to have its own unique details. Making sure the `getDetails()` method was implemented correctly in each subclass and worked with `super.toString()` also took some trial and error. Once I tested it and saw how the pieces connected, it became much clearer.

```
super(mod, mak, y);

// set number of doors
doors = d;

// set whether the car is electric
isElectric = e;

returns a string containing all vehicle and car details
ic String getDetails() {

// combine parent class string with car-specific details
return super.toString() + " |Doors: " + doors + " |Electric: " + isElectric;
```

At first I didn't know that the `toString` prints out the string that I have made in the vehicle class and it confused me on why it was printing everything out twice. Once I learned that it prints out the original `toString` I was able to make the layout look more professional.

How did you handle moments when you felt stuck or unsure of what to do next?

When I felt stuck or unsure, I handled it by focusing on one small part of the problem at a time instead of the whole program. I also asked ChatGPT and my teacher for help when something didn't make sense, especially with concepts like inheritance and abstract classes. Along with that, I re-read the problem requirements and tested small parts of my code to better understand what was going wrong. This helped me figure things out step by step and keep making progress.

Learning

Was there anything you learned that you think will help you with future challenges?

Something that will help me in future challenges is understanding abstract classes and using it for future projects. Although it confused me at first it allowed me to think deeper and understand how to use it when I have to for any new assignments.